A Mini-Project Report on

ProtectRequest

A Multi-Format Log Threat Analyzer

Submitted in partial fulfilment of the requirements of the
Honours in Cybersecurity
Bachelor of Engineering in Computer Engineering

Submitted by
Maric
Roll No.: [Roll Number]

Under the guidance of
[Guide Name]

Department of Computer Engineering
[College Name]
[University Name]
Academic Year 2026–27

Certificate

This is to certify that the mini-project entitled **“ProtectRequest: A Multi-Format Log Threat Analyzer”** is a bona fide work of **Maric** (Roll No. [Roll Number]), submitted in partial fulfilment of the requirements of the Honours in Cybersecurity under the Bachelor of Engineering in Computer Engineering of [University Name], during the academic year 2026–27.

[Guide Name]
Project Guide

[Head of Department]
Head of Department

[External Examiner]
External Examiner

[Principal]
Principal

Date: _____ Place: _____

Acknowledgement

I would like to thank my project guide, [Guide Name], for their guidance and feedback throughout this project, and the Head of the Department of Computer Engineering, [Head of Department], for providing the facilities and support needed to complete it. I am also grateful to the faculty of the Honours in Cybersecurity programme, whose teaching shaped the approach taken in this work, and to my family and friends for their encouragement.

Maric

Abstract

Server logs record nearly every attack against a system, but their volume and format make them hard to review by hand. This project presents **ProtectRequest**, a lightweight command-line tool that analyses Apache/Nginx access logs, Nginx error logs, Linux SSH authentication logs, Microsoft IIS logs and JSON logs, with automatic format detection. It combines regular-expression signatures for SQL injection, cross-site scripting, path traversal, command injection, Log4Shell, Shellshock, sensitive-file probes and scanner tools with sliding-window behavioural detection of request floods, brute-force logins, directory scanning and successful logins that follow a brute-force attempt. Suspicious IP addresses are enriched with offline GeoIP data and AbuseIPDB reputation scores (cached per IP), and each IP receives a 0–100 risk score. Results are delivered as a self-contained HTML report with charts, JSON and CSV exports, console summaries and a live monitoring mode that sends alerts to the console, a file or a webhook.

The project began with a security audit of an earlier script, which revealed a hard-coded API key and a stored cross-site scripting vulnerability in its own report; both are fixed in the new design, which treats all log content as untrusted. On a labelled synthetic dataset containing eight attackers among normal users, the tool detected and correctly ranked every attacker, flagged both cases of account compromise as Critical, and raised no alerts for normal users. It processed about 7,900 lines per second in testing, and is covered by 36 automated tests.

Keywords: log analysis, intrusion detection, web attacks, brute force, threat intelligence, risk scoring, Python.

Table of Contents

Chapter 1: Introduction **1**
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope 2
1.5 Origin of the Project 2
1.6 Organisation of the Report 2
Chapter 2: Literature Review **3**
2.1 Server Log Formats 3
2.2 Attack Classes Targeted 3
2.3 Existing Tools 3
2.4 Research Gap 4
Chapter 3: Requirements and Analysis **5**
3.1 Requirements 5
3.2 Security Audit of Version 1 6
Chapter 4: System Design **7**
4.1 Architecture 7
4.2 Data Model 7
4.3 Detection Design 8
4.4 Risk Scoring 9
4.5 Security of the Tool Itself 9
Chapter 5: Implementation **10**
5.1 Technology Stack 10
5.2 Project Structure 10
5.3 Key Implementation Details 10
5.4 Command-Line Interface 11
Chapter 6: Testing **13**
6.1 Strategy 13
6.2 Test Cases 13
6.3 Results 13
Chapter 7: Results and Discussion **15**
7.1 Evaluation Dataset 15
7.2 Detection Results 15
7.3 The HTML Report 16
7.4 Security Check: Stored XSS 19
7.5 Live Monitoring 19
7.6 Exports, Automation and IP Lookup 20
7.7 Performance 21
7.8 Discussion 21
Chapter 8: Conclusion and Future Scope **22**
8.1 Conclusion 22

8.2 Limitations..... 22
8.3 Future Scope..... 22
References..... 23
Appendix A: User Guide..... 24
A.1 Presentation Demo..... 24

List of Figures

Figure 4.1: Processing pipeline and supporting modules....7
Figure 5.1: Supported formats and detection rules listed by the CLI....12
Figure 6.1: Automated test suite: 36 tests passed....14
Figure 7.1: Console output for the web access log....16
Figure 7.2: SSH auth.log analysis: brute force and a compromised account....16
Figure 7.3: HTML report: summary, severity counts and activity timeline....17
Figure 7.4: HTML report: attack categories and top risky IPs....17
Figure 7.5: Flagged IPs, with one row expanded to show its evidence....18
Figure 7.6: Findings filtered for one IP: brute force followed by a successful login....18
Figure 7.7: XSS payload from a log shown as harmless text in the report....19
Figure 7.8: Watch mode raising alerts in real time....19
Figure 7.9: All five sample formats analysed together, with JSON and CSV export....20
Figure 7.10: Exit code 2 when high-risk IPs are found, for use in scripts and CI....20
Figure 7.11: IP lookup using the offline GeoIP database....20

List of Tables

Table 2.1: Attacks detected and their references 3
Table 2.2: Comparison with existing tools 4
Table 3.1: Software and hardware requirements 5
Table 3.2: Defects found in v1 and how v2 fixes them 6
Table 4.1: Modules 7
Table 4.2: Signature rules 8
Table 4.3: Behavioural rules and default thresholds 8
Table 5.1: Commands 11
Table 6.1: Selected test cases 13
Table 7.1: Ranked IPs for web_access.log and auth.log (1,299 events) 15

Chapter 1: Introduction

1.1 Background

Every web server, SSH daemon and application writes a log line for each request or login attempt it handles. These logs are one of the most reliable records of what has happened to a system, and they routinely contain the traces of attacks: SQL injection strings in URLs, password-guessing attempts against SSH, automated scanners probing for configuration files, and exploit payloads such as Log4Shell hidden in HTTP headers. NIST SP 800-92 describes log management as a core security practice precisely because logs are where such evidence first appears [5].

In practice, logs are rarely read. A modest server produces thousands of lines per hour, attack lines are mixed with normal traffic, payloads are often URL-encoded so they do not look suspicious at a glance, and some attacks, such as brute force or directory scanning, only become visible when many lines from the same source are counted over time. Large platforms (SIEMs such as Splunk, the Elastic stack or Wazuh) solve this but need significant setup. Students, small teams and incident responders working on a single machine need something lighter.

1.2 Problem Statement

To design and build a lightweight, offline-capable tool that reads common server log formats, detects attack activity both in individual requests and in behaviour over time, ranks the source IP addresses by risk, and presents the results in a form that a person can act on quickly — while itself being safe to run on attacker-controlled input.

1.3 Objectives

1. Parse the most common log formats (Apache/Nginx access, Nginx error, Linux SSH `auth.log`, Microsoft IIS W3C, JSON lines) and detect the format automatically.
2. Detect injection payloads (SQL injection, cross-site scripting, path traversal, command injection, Log4Shell, Shellshock), sensitive-file probes and known scanner tools.
3. Detect behavioural attacks with sliding time windows: request floods, web and SSH brute force, 404 directory scanning, and a successful login that follows a brute-force attempt.
4. Enrich suspicious IPs with country (offline GeoIP) and reputation (AbuseIPDB) data without wasting API quota.
5. Combine all evidence into a single 0–100 risk score per IP.
6. Produce a self-contained HTML report plus JSON and CSV exports, and support live monitoring with alerts.

7. Treat log content as untrusted input throughout, so the tool cannot be attacked through the logs it reads.

1.4 Scope

ProtectRequest is a detection and reporting tool. It does not block traffic (it is not a web application firewall) and it does not replace a full SIEM. It is aimed at single-server analysis, coursework and incident-response triage, and is written in Python so that it runs on Windows, Linux and macOS without a database or server component.

1.5 Origin of the Project

The project started from an existing script (referred to here as **v1**) that handled a single Combined-format file and flagged only requests to `/admin` and `/login`. A security review of v1 (Section 3.2) found several serious defects, including a hard-coded API key and a stored cross-site scripting vulnerability in its own report. Version 2, described in this report, is a redesign that fixes those defects and extends the tool substantially.

1.6 Organisation of the Report

Chapter 2 reviews log formats, the attacks the tool targets, and existing tools. Chapter 3 lists requirements and the v1 audit. Chapter 4 presents the design, Chapter 5 the implementation, and Chapter 6 the testing. Chapter 7 discusses results on a labelled sample dataset. Chapter 8 concludes and outlines future work.

Chapter 2: Literature Review

2.1 Server Log Formats

Common and Combined Log Format. Apache and Nginx write one line per request: client IP, identity, user, timestamp, request line, status code and response size; the Combined format adds the Referer and User-Agent headers [6][7]. This is the most widely used web log format.

Nginx error log. A separate free-text log recording failures such as missing files, rules that denied access, and malformed requests, with the client IP and request appended [7].

Linux authentication log. On Debian and Ubuntu systems `/var/log/auth.log` (on Red Hat systems `/var/log/secure`) records `sshd` events: failed and accepted passwords and keys, and attempts against user names that do not exist. Syslog timestamps omit the year.

W3C Extended Log File Format. Used by Microsoft IIS. Its columns are not fixed; each file declares them in a `#Fields:` header line [8].

JSON lines. Modern servers and proxies (Nginx with a JSON `log_format`, Caddy, Traefik, cloud load balancers) can write one JSON object per line. Key names vary between products.

2.2 Attack Classes Targeted

Table 2.1: Attacks detected and their references

Attack	What appears in the log	Reference
SQL injection	SQL keywords and quote tricks in query strings	OWASP A03:2021 [1]
Cross-site scripting	`<script>`, event handlers, `javascript:` URLs	OWASP A03:2021 [1], [15]
Path traversal / LFI	`../` sequences, `/etc/passwd`, `php://` wrappers	MITRE T1190 [2]
Command injection	Shell metacharacters followed by commands	OWASP A03:2021 [1]
Log4Shell	`${jndi:` lookups, often in User-Agent	CVE-2021-44228 [9]
Shellshock	`() { :; }` function definitions in headers	CVE-2014-6271 [10]
Brute force	Many failed logins from one source	MITRE T1110 [3]
Active scanning	Bursts of 404s, scanner User-Agents, probes for `.env`/`.git`	MITRE T1595 [4]

2.3 Existing Tools

Several mature tools work with logs. Table 2.2 summarises how they relate to the goal of this project.

Table 2.2: Comparison with existing tools

Tool	Approach	Difference from ProtectRequest
Fail2ban [14]	Regex filters on logs; bans IPs with the firewall	Bans per rule; no combined risk score, payload taxonomy or report
GoAccess [19]	Real-time web log analytics	Focus on traffic statistics, not attack detection
Wazuh / OSSEC [18]	Host intrusion detection with agents and a manager	Powerful but needs a server deployment
ModSecurity + CRS [17]	Web application firewall inline with the server	Blocks live requests; does not analyse existing log files
Splunk / Elastic	General log platforms with search and dashboards	Detection rules and dashboards must be built; heavy setup

2.4 Research Gap

There is room for a tool that sits between a hand-written `grep` and a full SIEM: one command, no server, several log formats, both payload and behaviour detection, IP reputation, a single ranked list of attackers, and a report that can be opened offline. ProtectRequest is designed to fill this gap.

Chapter 3: Requirements and Analysis

3.1 Requirements

Functional requirements

- FR1: Read one or more log files or whole folders, and detect each file's format automatically.
- FR2: Detect the attack classes in Table 2.1 and report each finding with severity, evidence and the original request.
- FR3: Track per-IP behaviour over sliding time windows to detect floods, brute force and scans.
- FR4: Look up country and reputation for suspicious IPs, with caching and a whitelist.
- FR5: Score each IP from 0 to 100 and assign a level (Low, Medium, High, Critical).
- FR6: Write HTML, JSON and CSV reports and print a console summary.
- FR7: Follow a log live and raise alerts to the console, a file or a webhook.
- FR8: Return a non-zero exit code when high-risk IPs are found, for automation.

Non-functional requirements

- **Security:** no secrets in source code; all log data escaped in output; private IPs never sent to third parties.
- **Portability:** pure Python 3.9+, works on Windows, Linux and macOS; paths resolved relative to the configuration file.
- **Offline operation:** full analysis without internet; reputation lookups optional.
- **Maintainability:** modular design so a new log format or rule is a single, isolated change; automated tests.
- **Performance:** process typical daily logs (tens of thousands of lines) in seconds.

Software and hardware requirements

Table 3.1: Software and hardware requirements

Item	Requirement
Operating system	Windows 10/11, Linux or macOS
Language	Python 3.9 or later (tested on 3.9 to 3.13)
Libraries	`requests`, `geoip2`, `colorama`; `pytest` for tests
Data	MaxMind GeoLite2-Country database (optional) [12]
API	AbuseIPDB API key (optional, free tier) [13]
Hardware	Any machine that runs Python; ~40 MB RAM in testing

3.2 Security Audit of Version 1

Before redesigning, the original script was reviewed as if it were third-party code. Table 3.2 lists the defects found. Two of them are security vulnerabilities in the tool itself, which is especially serious for a security tool.

Table 3.2: Defects found in v1 and how v2 fixes them

v1 defect	Impact	v2 fix
AbuseIPDB API key hard-coded in `config.py` and `config.ini`	Secret exposed in the repository and its history	Key read from the `ABUSEIPDB_API_KEY` environment variable; old key must be revoked
HTML report built with unescaped log data	Stored XSS: a crafted URL in a log runs script when the report is opened	Every value passed through `html.escape`; Content-Security-Policy added
API called once per log line	Free quota (1,000/day) exhausted quickly; slow	One lookup per IP, cached on disk with a time-to-live
“Requests per minute” was a running total	Normal users flagged after a long day	Real sliding window per IP
Docker-only `/app/...` paths	Crash on Windows and outside Docker	Paths resolved relative to the configuration file
Only `/admin` and `/login` checked	Almost all attacks missed	15+ rules across 6 log formats
Whitelist and status-code settings never used	Configuration had no effect	Both implemented

Chapter 4: System Design

4.1 Architecture

ProtectRequest is a pipeline of small stages. Each stage depends only on the data type produced by the stage before it, which keeps modules independent and testable (Figure 4.1).

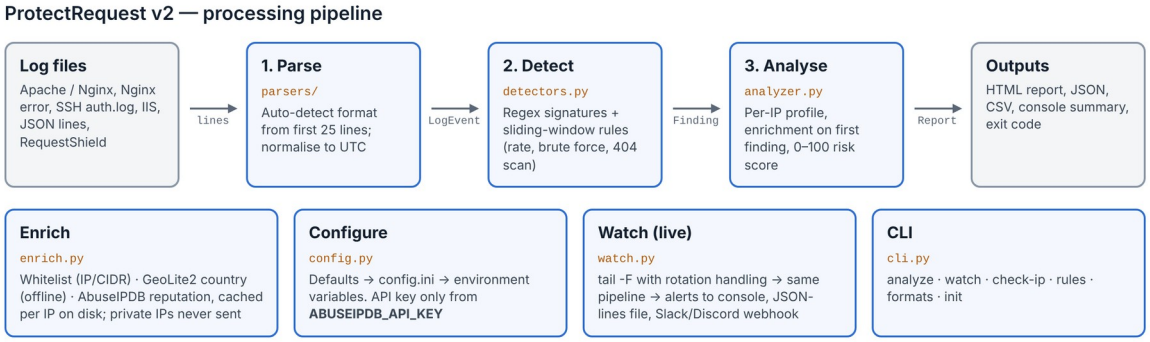


Figure 4.1: *Processing pipeline and supporting modules*

Table 4.1: Modules

Module	Responsibility
`parsers/`	One parser per format; each turns a raw line into a `LogEvent`. `detect_format()` tries all parsers on the first 25 lines and picks the best.
`models.py`	Shared types: `LogEvent`, `Finding`, `Severity`.
`detectors.py`	`SignatureDetector` (regex rules) and stateful sliding-window detectors.
`enrich.py`	Whitelist, GeoIP lookup, AbuseIPDB client with memory and disk cache.
`analyzer.py`	Runs events through detectors, builds per-IP profiles, scores them, produces a `Report`.
`html_report.py`, `output.py`	HTML report with inline SVG charts; JSON, CSV and console output.
`watch.py`	Live tail with log-rotation handling, alert de-duplication and webhooks.
`config.py`, `cli.py`	Layered configuration and the command-line interface.

4.2 Data Model

A **LogEvent** is one normalised log line whatever its source: source format, line number, IP, timestamp (converted to UTC), method, path, status, size, user agent, user name, action (for example `auth_failed`) and message. A **Finding** records one detection: rule identifier, category, severity, description, IP, the event that triggered it, and the matched evidence. An **IPProfile** aggregates everything about one IP: request count, first and last seen, status codes, paths, user agents, user names tried, its findings, enrichment data, score and level.

4.3 Detection Design

Signature detection

Signature rules are regular expressions applied to the request path and the User-Agent. Before matching, the path is URL-decoded twice and lower-cased, so double-encoded payloads such as `%252e%252e%252f` (which decodes to `../`) are still recognised. Rules that describe the client rather than the request (scanner and scripted-client User-Agents) are reported once per IP to avoid flooding the report.

Table 4.2: Signature rules

Rule	Severity	Examples matched
SQLI	High	`UNION SELECT`, `' OR '1'='1`, `SLEEP(5)`, `information_schema`
XSS	High	`<script`, `onerror=`, `javascript:`, `document.cookie`
TRAVERSAL	High	`../`, `/etc/passwd`, `php://filter`, `/proc/self/`
CMDI	Critical	`;cat`, `|id`, `$(curl …)`, `/bin/sh`
LOG4SHELL / SHELLSHOCK	Critical	`${jndi:`, `() { :; }`
SENSITIVE_FILE	Medium	`/.env`, `/.git/`, `wp-config.php`, `.bak`, `.sql`
SCANNER_UA	Medium	sqlmap, nikto, gobuster, nuclei, wpscan, hydra
ADMIN_PROBE	Low	`/wp-login.php`, `/phpmyadmin`, `/admin`
BAD_METHOD	Low–Medium	TRACE, CONNECT, PUT, DELETE, PROPFIND

Behavioural detection with sliding windows

Each behavioural detector keeps, per IP, a double-ended queue of event timestamps. When a new event arrives, timestamps older than the window are removed from the front, so the queue length is exactly the number of events in the last *N* seconds. This replaces the running total used in v1.

To avoid one finding per line during a long attack, alerts **escalate**: the first fires at the threshold, the next when the count doubles (5, 10, 20, 40, …). When the window empties below the threshold, escalation resets. Long or intense attacks therefore produce a few findings and a higher score, rather than hundreds of duplicates.

Table 4.3: Behavioural rules and default thresholds

Rule	Condition (configurable)	Severity
RATE_LIMIT	More than 120 requests from one IP in 60 s	High
BRUTE_FORCE	5 or more failed SSH or web logins in 300 s	High

Rule	Condition (configurable)	Severity
BRUTE_FORCE_SUCCESS	A successful login from an IP previously flagged for brute force	Critical
SCAN_404	15 or more 404 responses in 120 s	Medium

For web logins the tool uses a heuristic: a `401`/`403` response, or a `POST` to a login URL answered with `200` (the form shown again), counts as a failure; a `302`/`303` redirect after a `POST` counts as a success.

4.4 Risk Scoring

Each severity carries points (info 1, low 3, medium 8, high 15, critical 30). For every distinct rule an IP triggered, the score adds the rule's points multiplied by a logarithmic factor of how often it fired, so repetition raises the score without letting one noisy rule dominate:

```
per_rule  = points(severity) x (1 + log2(hits))
ip_score  = sum(per_rule) + abuse_confidence x 0.5 + geo_points (if risky
country)
if any finding is Critical: ip_score = max(ip_score, 70)
ip_score  = min(ip_score, 100)
level     = Critical >= 70 > High >= 40 > Medium >= 15 > Low > 0
```

The rule that a single Critical finding raises the score to at least 70 reflects that one confirmed remote-code-execution attempt or account compromise matters more than many minor events.

4.5 Security of the Tool Itself

- **Untrusted input:** log lines are written by attackers. Every value placed in the HTML report is escaped, and the report sets a strict Content-Security-Policy that forbids external resources [15][16].
- **Secrets:** the API key is read from an environment variable and never written to disk by the tool.
- **Privacy:** private, loopback and reserved addresses are never sent to AbuseIPDB.
- **Fail-safe:** after three API errors or an HTTP 429 response, lookups stop and analysis continues.
- **No code execution:** the tool only reads files and never evaluates log content.

Chapter 5: Implementation

5.1 Technology Stack

The tool is written in Python using only three runtime libraries: `requests` for the AbuseIPDB API, `geoip2` for the MaxMind database, and `colorama` for coloured output on Windows terminals. Charts in the HTML report are drawn as inline SVG by Python code, so the report needs no JavaScript library or internet connection. Tests use `pytest`. The package is about 2,200 lines of Python, with 208 lines of tests and 255 lines of demo tooling.

5.2 Project Structure

```
ProtectRequest/
├── protectrequest/          the package
│   ├── parsers/             http, ssh, iis, jsonl (6 formats)
│   ├── models.py  detectors.py  enrich.py  analyzer.py
│   ├── html_report.py  output.py  watch.py  config.py  cli.py  colors.py
├── tests/                   36 automated tests
├── samples/                 labelled demo logs in five formats
├── tools/                   make_samples.py, demo_feed.py
├── legacy/                  the original v1 script, kept for reference
├── run.py  demo.bat  config.ini  requirements.txt  pyproject.toml  README.md
```

5.3 Key Implementation Details

Format auto-detection

Each parser exposes `matches(line)`. The first 25 non-empty lines of a file are given to every parser, and the one that accepts the most lines is chosen. A fresh parser instance is created per file, which lets the IIS parser keep state: it reads the `#Fields:` header and maps later lines by column name. All timestamps are converted to naive UTC so that events from different files and time zones can be compared.

Sliding window

The core of the behavioural detectors is shown below. `deque` gives constant-time removal from the front.

```
def add(self, ip, ts):
    q = self.hits[ip]
    q.append(ts)
    while q and ts - q[0] > self.span:   # drop events older than the window
        q.popleft()
    if len(q) < self.threshold:
        self.next_alert.pop(ip, None)     # burst over: reset escalation
    return len(q)

def should_alert(self, ip, count):
    if count >= self.next_alert.get(ip, self.threshold):
        self.next_alert[ip] = count * 2   # next alert when the count doubles
        return True
    return False
```

Reputation caching

AbuseIPDB results are cached per IP in memory and in a JSON file with a configurable time-to-live (24 hours by default). Lookups happen only for IPs that triggered at least one finding, and only once per IP; GeoIP, which is local and free, is applied to every IP.

Live monitoring

Watch mode follows a file like `tail -F`: it reads new lines as they are written, and if the file is replaced (log rotation) or truncated it re-opens it. Each line goes through the same parser and detectors as batch mode. To prevent alert storms, the same rule for the same IP alerts at most once per minute. Alerts go to the console and optionally to a JSON-lines file and a webhook; the payload carries both `text` and `content` fields so it works with Slack- and Discord-style webhooks.

5.4 Command-Line Interface

Table 5.1: Commands

Command	Purpose
`analyze FILES…`	Analyse files or folders; options for format, JSON/CSV output, minimum score, `--skip-unchanged`, `--fail-on LEVEL`
`watch FILE`	Follow a log live; `--webhook`, `--alert-file`, `--min-severity`, `--report` on exit
`check-ip IP…`	GeoIP and AbuseIPDB lookup for given addresses
`rules`, `formats`	List detection rules and supported formats
`init`	Write a commented default `config.ini`

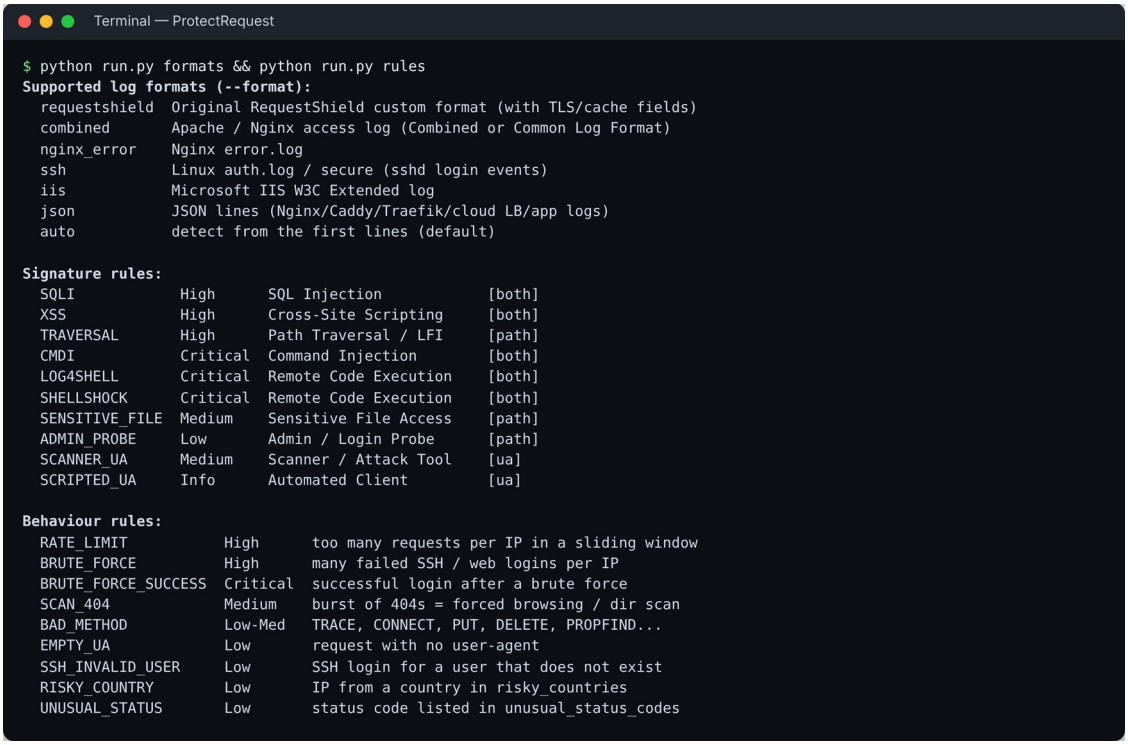


Figure 5.1: *Supported formats and detection rules listed by the CLI*

Chapter 6: Testing

6.1 Strategy

Testing covered three levels. **Unit tests** check each parser and each signature in isolation. **Behaviour tests** feed timed events to the analyzer to check sliding windows, escalation and the brute-force-then-success rule. **End-to-end tests** run the CLI on the sample logs and check the produced files and exit codes. Equal attention was given to **false positives**: URLs that merely contain words like *select*, *union* or *admin* must not be flagged.

6.2 Test Cases

Table 6.1: Selected test cases

ID	Test	Input	Expected result
T1	Combined parser	Standard Apache line	All fields extracted; timestamp in UTC
T2	Time zone	Line with `+0530` offset	Converted to UTC
T3	IIS parser	`#Fields:` header then data	Columns mapped by header
T4	Auto-detect	Each of five sample files	Correct format chosen
T5	Double encoding	`%252e%252e%252f`	Decoded to `../`
T6	Signatures	SQLi, XSS, traversal, CMDi, `.env`, `.git`	Matching rule fires
T7	False positives	`/selection?from=menu`, `/blog/union-station-review`	No Medium+ finding
T8	Rate window	45 requests in 45 s, limit 10/60 s	Alerts at 11, 22, 44
T9	Slow traffic	50 requests, 10 s apart	No rate alert
T10	Brute force → success	6 × 401 then 302 on `/login`	Critical level
T11	Whitelist	Attack from whitelisted CIDR	Ignored
T12	XSS-safe report	`<script>alert('pwned')` in a log	Escaped in HTML
T13	CLI end-to-end	`--json --csv --fail-on critical`	Files written; exit code 2
T14	Config precedence	Key in file and in environment	Environment wins

6.3 Results

All 36 tests pass (Figure 6.1). The suite was also run on Python 3.9, the oldest supported version, with the same result.

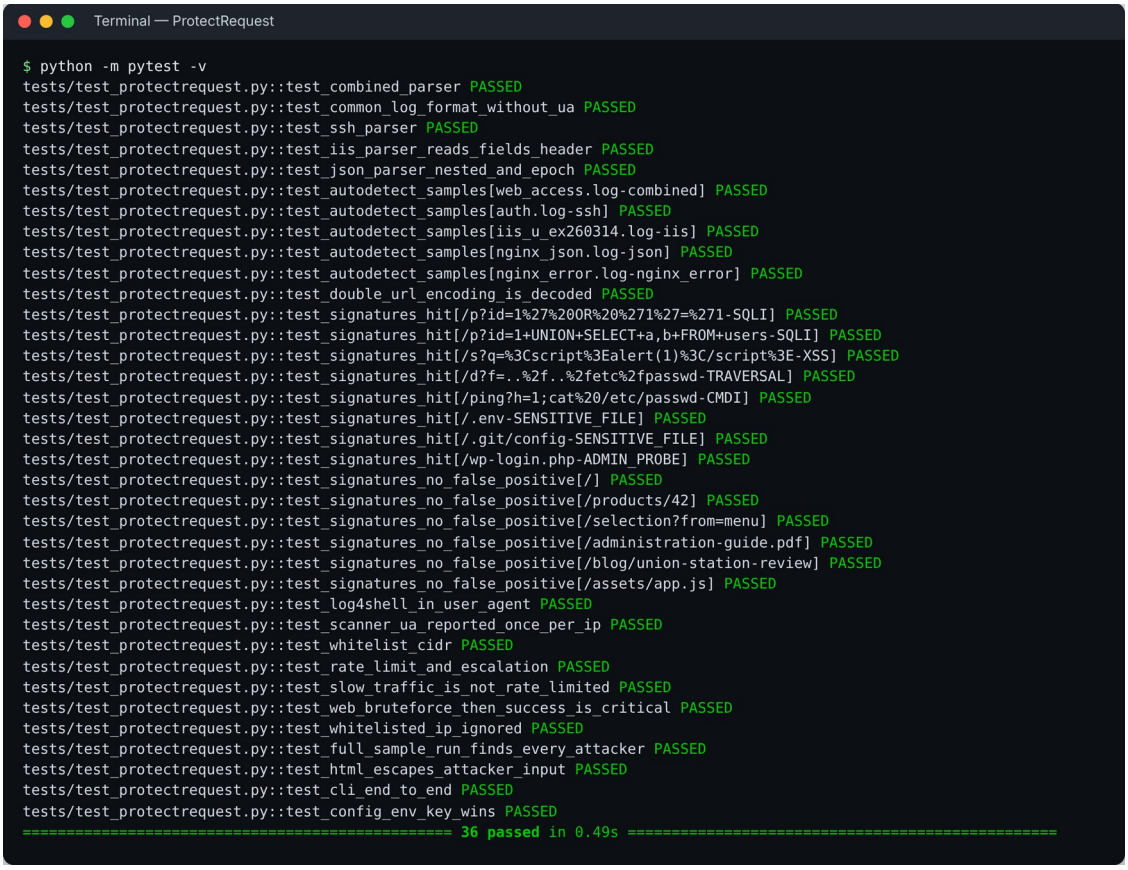


Figure 6.1: *Automated test suite: 36 tests passed*

Chapter 7: Results and Discussion

7.1 Evaluation Dataset

Real production logs contain personal data, so a labelled synthetic dataset was generated (`tools/make_samples.py`). It contains about 900 normal browsing requests from five users over six hours, plus eight planted attackers whose behaviour is known in advance. All IP addresses come from the ranges reserved for documentation by RFC 5737 [11], so no real person or organisation is named. For that reason the country column shows *Private / Reserved* in the results, and no AbuseIPDB lookups are made. The same events are also written in IIS and JSON formats, together with an SSH `auth.log` and an Nginx error log.

7.2 Detection Results

Table 7.1 shows the result of analysing the web access log and the SSH log together. Every planted attacker was detected and ranked, and none of the five normal users was flagged.

Table 7.1: Ranked IPs for web_access.log and auth.log (1,299 events)

IP	Score	Level	Planted behaviour
192.0.2.150	100	Critical	Command injection and Log4Shell
203.0.113.66	88	Critical	sqlmap, 20 SQL injection requests
198.51.100.200	78	Critical	XSS and path traversal
198.51.100.99	76	Critical	Web login brute force, then successful login
192.0.2.44	70	Critical	SSH brute force, then successful login
203.0.113.77	67	High	gobuster directory scan, `.env` and `.git` probes
203.0.113.5	54	High	SSH brute force against 20 user names
192.0.2.201	31	Medium	HTTP flood, about 5 requests per second
203.0.113.120	27	Medium	Unusual methods (TRACE, PUT…) with no User-Agent

The ordering matches intuition: attacks that could lead to code execution or account takeover rank highest, reconnaissance ranks next, and noisy but less dangerous behaviour such as a flood ranks Medium. Both brute-force-then-success cases were correctly raised to Critical, which is the most important single signal in the dataset because it indicates a probable compromise.



Figure 7.1: *Console output for the web access log*



Figure 7.2: *SSH auth.log analysis: brute force and a compromised account*

7.3 The HTML Report

The report opens in any browser without an internet connection. It shows summary figures, findings by severity, activity per hour, attack categories and the top IPs, followed by a

searchable table of flagged IPs (each row expands to show the rules hit, time span, status codes, top paths and user agents) and a searchable list of all findings.

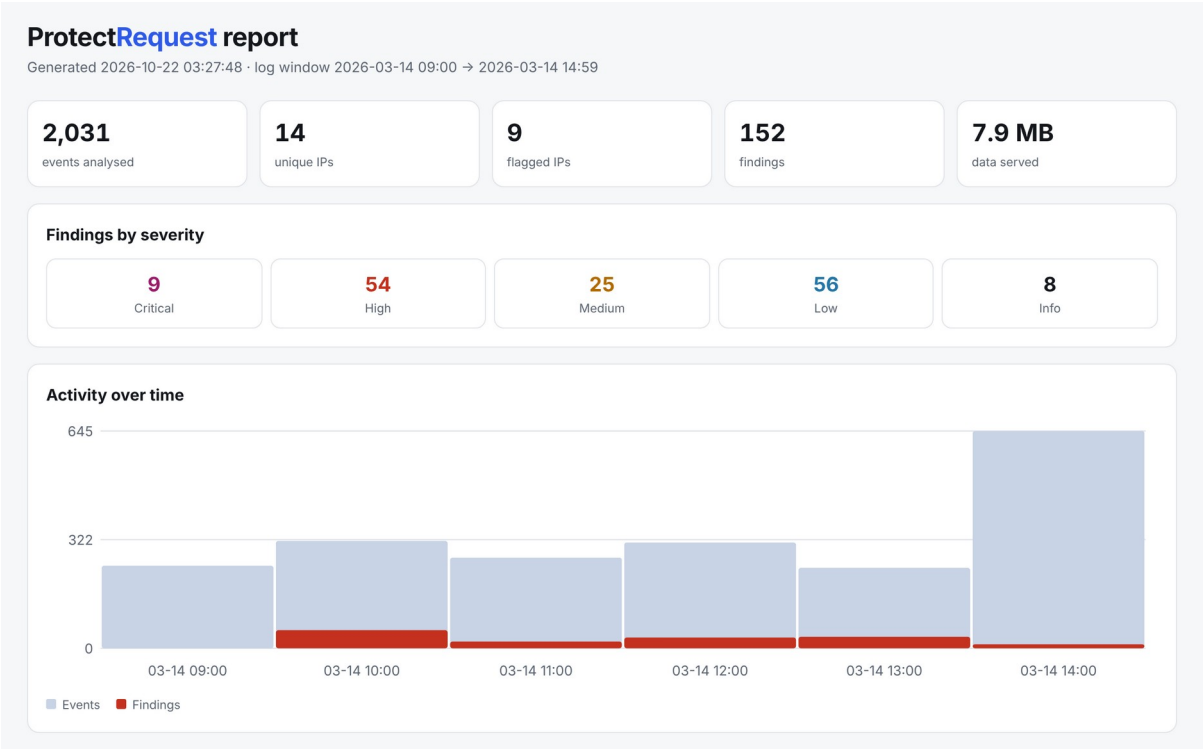


Figure 7.3: *HTML report: summary, severity counts and activity timeline*

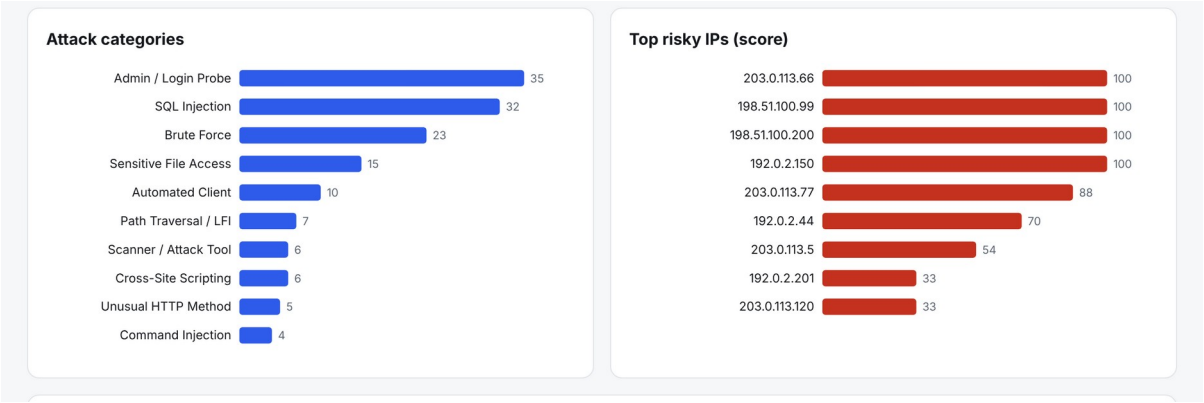
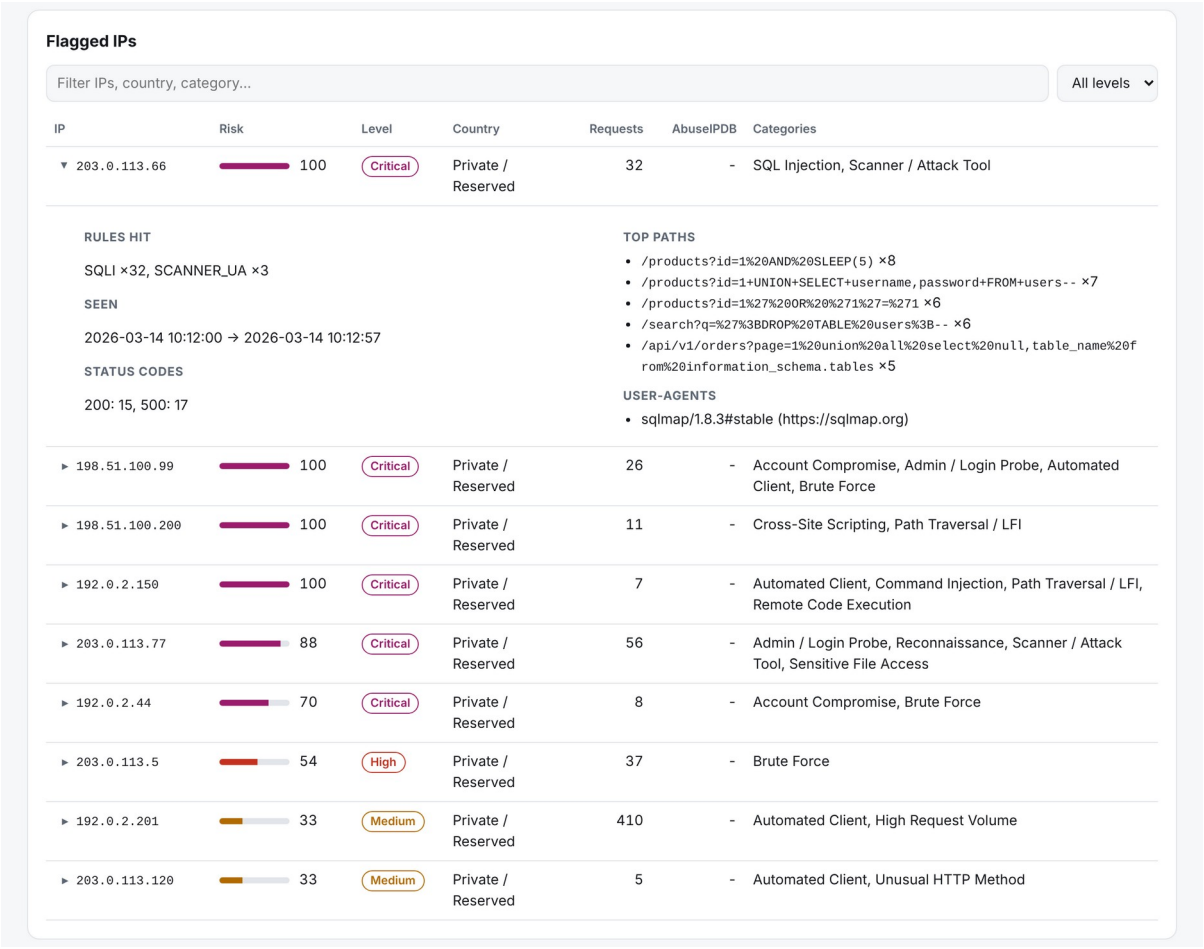


Figure 7.4: *HTML report: attack categories and top risky IPs*



Flagged IPs

Filter IPs, country, category... | All levels

IP	Risk	Level	Country	Requests	AbuseIPDB	Categories
▾ 203.0.113.66	100	Critical	Private / Reserved	32	-	SQL Injection, Scanner / Attack Tool
▸ 198.51.100.99	100	Critical	Private / Reserved	26	-	Account Compromise, Admin / Login Probe, Automated Client, Brute Force
▸ 198.51.100.200	100	Critical	Private / Reserved	11	-	Cross-Site Scripting, Path Traversal / LFI
▸ 192.0.2.150	100	Critical	Private / Reserved	7	-	Automated Client, Command Injection, Path Traversal / LFI, Remote Code Execution
▸ 203.0.113.77	88	Critical	Private / Reserved	56	-	Admin / Login Probe, Reconnaissance, Scanner / Attack Tool, Sensitive File Access
▸ 192.0.2.44	70	Critical	Private / Reserved	8	-	Account Compromise, Brute Force
▸ 203.0.113.5	54	High	Private / Reserved	37	-	Brute Force
▸ 192.0.2.201	33	Medium	Private / Reserved	410	-	Automated Client, High Request Volume
▸ 203.0.113.120	33	Medium	Private / Reserved	5	-	Automated Client, Unusual HTTP Method

RULES HIT

SQLI ×32, SCANNER_UA ×3

SEEN

2026-03-14 10:12:00 → 2026-03-14 10:12:57

STATUS CODES

200: 15, 500: 17

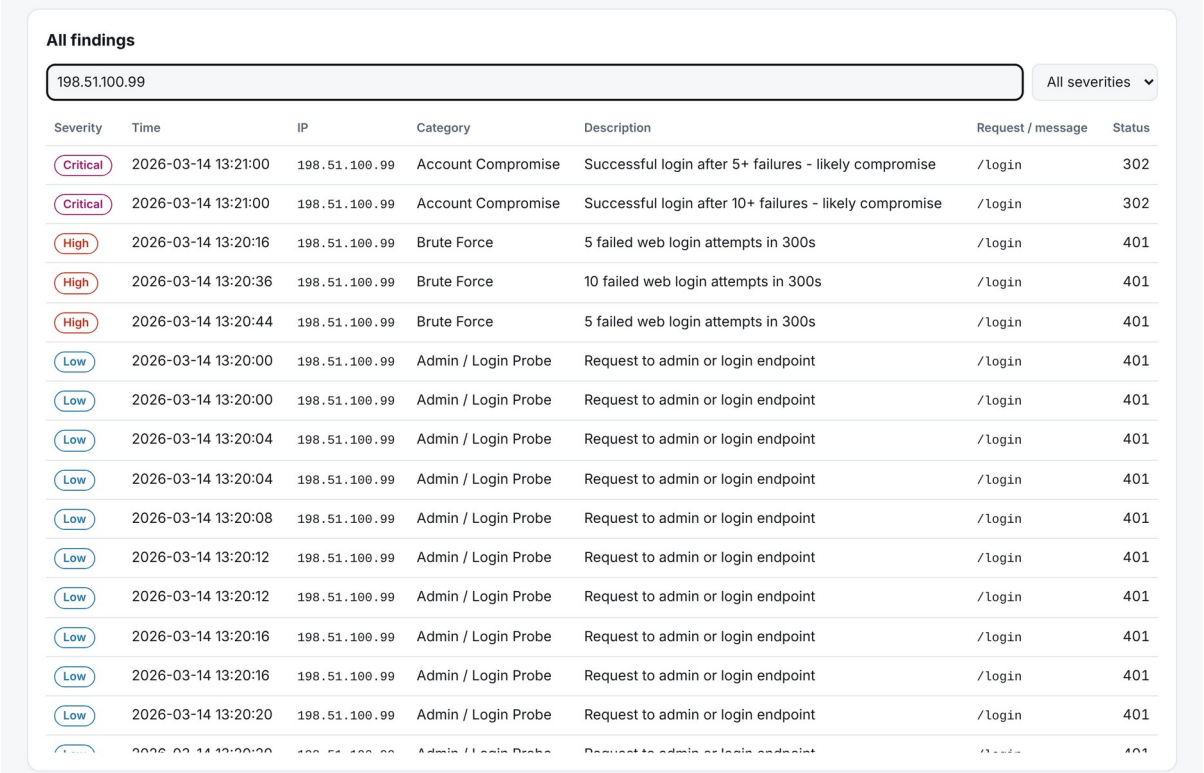
TOP PATHS

- `/products?id=1%20AND%20SLEEP(5)` ×8
- `/products?id=1+UNION+SELECT+username,password+FROM+users--` ×7
- `/products?id=1%27%20OR%20%271%27=%271` ×6
- `/search?q=%27%3BDROP%20TABLE%20users%3B--` ×6
- `/api/v1/orders?page=1%20union%20all%20select%20null,table_name%20from%20information_schema.tables` ×5

USER-AGENTS

- sqlmap/1.8.3#stable (https://sqlmap.org)

Figure 7.5: *Flagged IPs, with one row expanded to show its evidence*



All findings

198.51.100.99 | All severities

Severity	Time	IP	Category	Description	Request / message	Status
Critical	2026-03-14 13:21:00	198.51.100.99	Account Compromise	Successful login after 5+ failures - likely compromise	/login	302
Critical	2026-03-14 13:21:00	198.51.100.99	Account Compromise	Successful login after 10+ failures - likely compromise	/login	302
High	2026-03-14 13:20:16	198.51.100.99	Brute Force	5 failed web login attempts in 300s	/login	401
High	2026-03-14 13:20:36	198.51.100.99	Brute Force	10 failed web login attempts in 300s	/login	401
High	2026-03-14 13:20:44	198.51.100.99	Brute Force	5 failed web login attempts in 300s	/login	401
Low	2026-03-14 13:20:00	198.51.100.99	Admin / Login Probe	Request to admin or login endpoint	/login	401
Low	2026-03-14 13:20:00	198.51.100.99	Admin / Login Probe	Request to admin or login endpoint	/login	401
Low	2026-03-14 13:20:04	198.51.100.99	Admin / Login Probe	Request to admin or login endpoint	/login	401
Low	2026-03-14 13:20:04	198.51.100.99	Admin / Login Probe	Request to admin or login endpoint	/login	401
Low	2026-03-14 13:20:08	198.51.100.99	Admin / Login Probe	Request to admin or login endpoint	/login	401
Low	2026-03-14 13:20:12	198.51.100.99	Admin / Login Probe	Request to admin or login endpoint	/login	401
Low	2026-03-14 13:20:12	198.51.100.99	Admin / Login Probe	Request to admin or login endpoint	/login	401
Low	2026-03-14 13:20:16	198.51.100.99	Admin / Login Probe	Request to admin or login endpoint	/login	401
Low	2026-03-14 13:20:16	198.51.100.99	Admin / Login Probe	Request to admin or login endpoint	/login	401
Low	2026-03-14 13:20:20	198.51.100.99	Admin / Login Probe	Request to admin or login endpoint	/login	401

Figure 7.6: *Findings filtered for one IP: brute force followed by a successful login*

7.4 Security Check: Stored XSS

To confirm the v1 vulnerability is fixed, a log was created containing `<script>alert(…)</script>` and an `<img onerror>` payload in request URLs. In the generated report the payloads appear as text (Figure 7.5) and no script runs; an automated browser check confirmed that no alert dialog was raised.

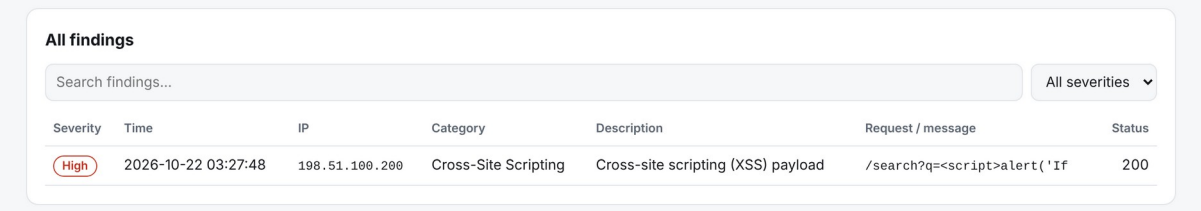


Figure 7.7: *XSS payload from a log shown as harmless text in the report*

7.5 Live Monitoring

In watch mode, attacks appended to a live log produced alerts within the polling interval (0.5 s in the demo). Figure 7.6 shows the alerts for a scripted sequence of attacks, ending with a Critical alert for a successful login after repeated failures.



Figure 7.8: *Watch mode raising alerts in real time*

7.6 Exports, Automation and IP Lookup

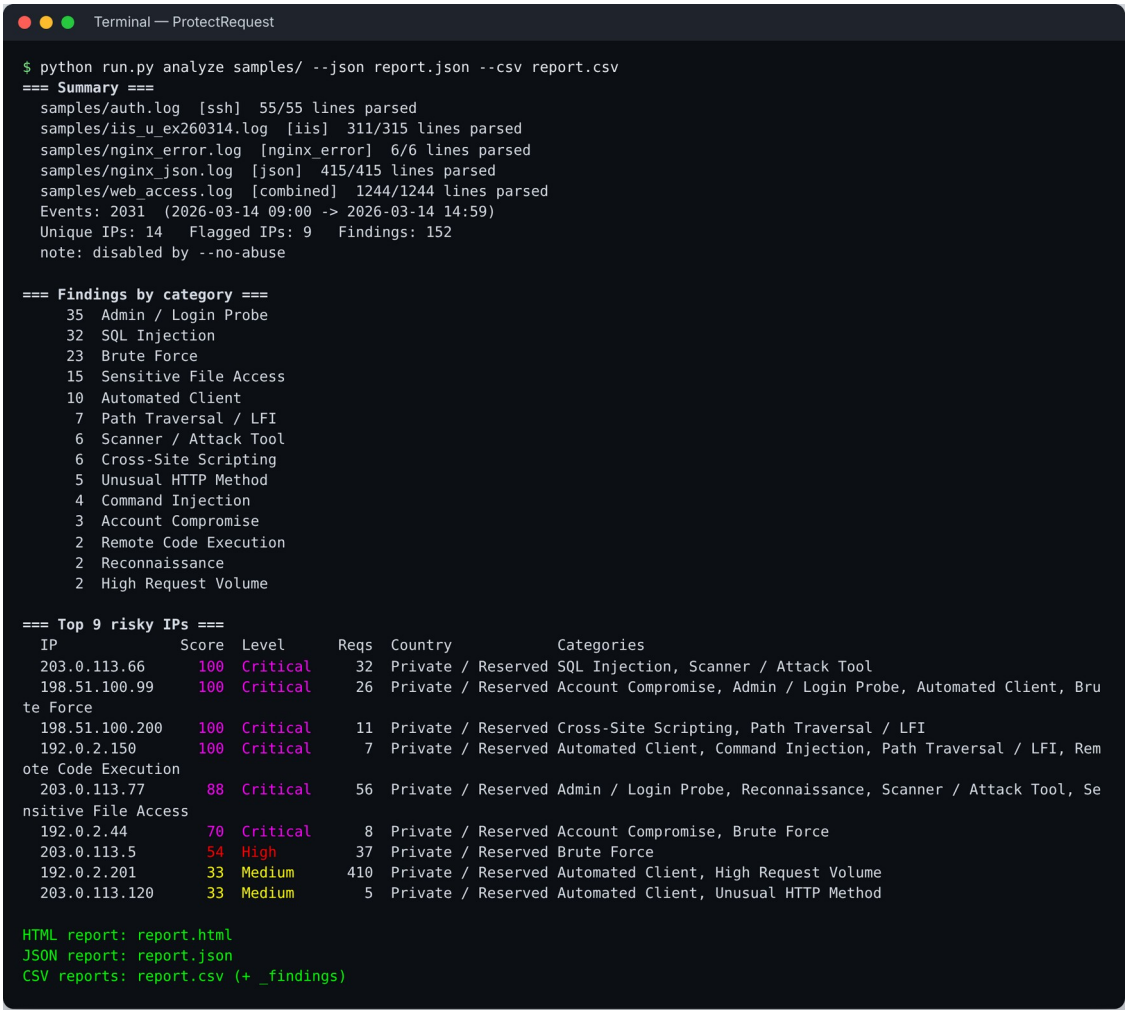


Figure 7.9: *All five sample formats analysed together, with JSON and CSV export*



Figure 7.10: *Exit code 2 when high-risk IPs are found, for use in scripts and CI*

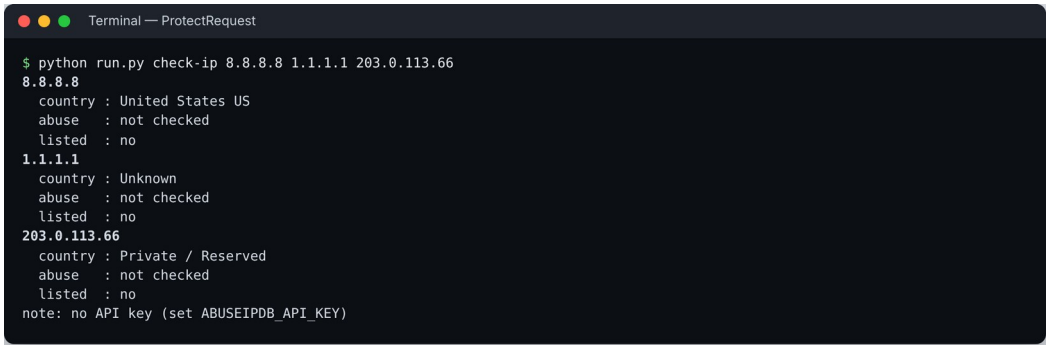


Figure 7.11: *IP lookup using the offline GeoIP database*

7.7 Performance

A 124,400-line file (the web sample repeated 100 times, so it is attack-heavy) was analysed three times on a 2-vCPU Linux cloud machine with Python 3.13. Each run took 15.4–16.3 s, about 7,900 lines per second, with a peak memory use of about 37 MB. A typical day of logs for a small server is therefore analysed in seconds. Speed was not a primary design goal; Section 8.3 notes how it could be improved.

7.8 Discussion

The combination of signature and behavioural detection is what makes the results useful. Signatures alone would have missed the brute-force and flood IPs, whose individual requests look normal; behaviour alone would have missed single-request exploits such as Log4Shell. Scoring at the IP level, rather than listing every finding, turns about a hundred findings into nine ranked addresses that an analyst can review in order.

Chapter 8: Conclusion and Future Scope

8.1 Conclusion

ProtectRequest v2 meets the objectives set in Section 1.3. It parses six log formats with automatic detection, recognises injection payloads, scanners and sensitive-file probes, detects floods, brute force and scans with sliding windows, flags probable account compromise, enriches IPs with country and reputation data, and ranks them with a single score. It produces an offline HTML report, JSON and CSV exports, and live alerts. On the labelled dataset every planted attacker was found and no normal user was flagged. The redesign also removed two security vulnerabilities from the original script — a leaked API key and a stored XSS in the report — which shows the importance of treating log data as hostile input.

8.2 Limitations

- Regular-expression signatures can be evaded by sufficiently obfuscated payloads; the tool is not a replacement for a web application firewall.
- Web login success and failure are inferred from status codes and may be wrong for applications with unusual behaviour.
- Syslog lines carry no year, so the current year is assumed.
- The evaluation used a synthetic dataset; performance on real-world traffic has not yet been measured.
- Scores are heuristics and should be reviewed before an IP is blocked.

8.3 Future Scope

- **Automatic response:** push Critical IPs to the firewall or Fail2ban, with an allow-list safety check.
- **Community rules:** load Sigma detection rules in addition to the built-in signatures.
- **Anomaly detection:** learn normal traffic per IP and per path, and flag deviations.
- **More sources:** Windows Event Log, firewall logs and cloud audit trails.
- **Performance:** parallel parsing across files and pre-compiled combined patterns.
- **Evaluation:** test on public labelled datasets to measure precision and recall.

References

[1] OWASP Foundation, “A03:2021 – Injection,” OWASP Top 10:2021. https://owasp.org/Top10/A03_2021-Injection/

[2] MITRE, “T1190: Exploit Public-Facing Application,” ATT&CK. https://attack.mitre.org/techniques/T1190/

[3] MITRE, “T1110: Brute Force,” ATT&CK. https://attack.mitre.org/techniques/T1110/

[4] MITRE, “T1595: Active Scanning,” ATT&CK. https://attack.mitre.org/techniques/T1595/

[5] K. Kent and M. Souppaya, “Guide to Computer Security Log Management,” NIST Special Publication 800-92, 2006.

[6] The Apache Software Foundation, “Log Files,” Apache HTTP Server 2.4 Documentation. https://httpd.apache.org/docs/2.4/logs.html

[7] NGINX, “Module ngx_http_log_module.” https://nginx.org/en/docs/http/ngx_http_log_module.html

[8] P. M. Hallam-Baker and B. Behlendorf, “Extended Log File Format,” W3C Working Draft WD-logfile-960323, 1996.

[9] NIST National Vulnerability Database, “CVE-2021-44228 (Apache Log4j2 JNDI).” https://nvd.nist.gov/vuln/detail/CVE-2021-44228

[10] NIST National Vulnerability Database, “CVE-2014-6271 (GNU Bash ‘Shellshock’).” https://nvd.nist.gov/vuln/detail/CVE-2014-6271

[11] J. Arkko, M. Cotton and L. Vegoda, “IPv4 Address Blocks Reserved for Documentation,” RFC 5737, IETF, 2010.

[12] MaxMind, “GeoLite2 Free Geolocation Data.” https://dev.maxmind.com/geoip/geolite2-free-geolocation-data

[13] AbuseIPDB, “APIv2 Documentation.” https://docs.abuseipdb.com/

[14] Fail2ban project. https://github.com/fail2ban/fail2ban

[15] OWASP Foundation, “Cross Site Scripting Prevention Cheat Sheet.” https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

[16] W3C, “Content Security Policy Level 3.” https://www.w3.org/TR/CSP3/

[17] OWASP Foundation, “OWASP ModSecurity Core Rule Set.” https://coreruleset.org/

[18] Wazuh, “Wazuh Documentation.” https://documentation.wazuh.com/

[19] GoAccess, “GoAccess – Visual Web Log Analyzer.” https://goaccess.io/

Appendix A: User Guide

```
pip install -r requirements.txt

# optional: AbuseIPDB key (PowerShell)
$env:ABUSEIPDB_API_KEY = "your-key"

python run.py analyze samples/ --no-abuse            # try on demo logs
python run.py analyze access.log auth.log --json out.json --csv out.csv
python run.py analyze u_ex260314.log -f iis --min-score 40
python run.py analyze access.log --skip-unchanged --fail-on high  # CI
python run.py watch /var/log/auth.log --webhook URL --report live.html
python run.py check-ip 203.0.113.5
python -m pytest -q                                  # run tests
```

A.1 Presentation Demo

`demo.bat` (Windows) runs ten steps in order with a pause between each: setup, formats and rules, tests, a web log report, SSH brute force, all formats with exports, the XSS-safety check, live watch mode in a second window while attacks are fed in, the CI exit code, and an IP lookup. It locates a working Python installation automatically.